BOXSPLOIT LABS / PRODUCT DOCUMENTATION

PANBOX

Product & operator guide

Self-hosted AppSec control plane. Setup, CI connectivity, access control, operations, and honest pilot limits.

VERSION 0.1 · 17 OCTOBER 2026 · PILOT

© 2026 BOXSPLOIT LABS

Contents

1. Overview and architecture
2. Requirements and operating systems
3. Local installation on Debian/Linux/macOS
4. Local installation on Windows
5. SCM repository, native agent, and CLI
6. Public SCM to private PANBOX
7. RBAC and repository access
8. API, access, and scan keys
9. Dashboard, findings, gate, and reports
10. Operations, security, and resource sizing
11. Troubleshooting
12. Pilot limits and next steps

This guide describes implemented pilot behavior. Roadmap items are labelled separately.

© 2026 BOXSPLOIT LABS

Guide version: 0.1 (pilot, 17 October 2026). PANBOX is a self-hosted AppSec control plane by BOXSPLOIT LABS. This guide describes the implemented features, setup, and pilot limits. The web page and PDF are product documentation, not vulnerability reports.

1. Overview and architecture

PANBOX receives native scans from the panbox agent, stores snapshots per repository and branch, evaluates a quality gate, and presents posture, findings, scan history, technical exports, RBAC, and audit events. A Go server serves the API and React UI; PostgreSQL stores state. The agent runs on developer machines or CI runners, not in the dashboard container.

High-level infrastructure and data flow:

```
Git host + CI --checkout--> protected runner + PANBOX agent
                                |-- VACCIN: source rules
                                |-- ENETRON: pinned packages --HTTPS--> OSV data
                                +-- findings bundle + scan key --> PANBOX Go API
                                                                     |--> PostgreSQL
Operator browser --loopback/HTTPS--> React UI -----------------+
```

The private PANBOX host serves the UI and API and stores data in PostgreSQL; it does not scan source code. The runner sends package names and versions to the advisory API, while the normalized scan bundle goes to PANBOX. A local runner, an SSH relay, or later artifact import can bridge public CI to the private host; only the first two produce a synchronous CI gate. See docs/architecture.md for trust boundaries and each network route.

VACCIN is PANBOX's own narrow source-rule engine for selected Python, JavaScript/TypeScript, and Go patterns. ENETRON is PANBOX's own pinned-dependency parser and advisory matcher for npm lockfiles v2/v3, requirements.txt, and go.mod. ENETRON retrieves vulnerability data from an OSV-compatible HTTPS API; OSV is an advisory source, not a scanner executable. The agent does not execute Semgrep, Grype, or another scanner. Coverage is intentionally limited and does not include broad dataflow analysis.

Flow: Git source -> PANBOX agent -> native rules and dependency matching -> normalized bundle -> PANBOX API -> PostgreSQL -> dashboard and gate decision. A failed or incomplete module stays partial/unverified and does not erase the last successful snapshot. A missing supported manifest marks ENETRON as error rather than passing silently.

The current distinct value is a small single-tenant workflow with PANBOX-owned detection logic, repository and branch history, an explicit unverified state, scoped RBAC and keys, and private-network delivery for public SCM repositories. This is not a claim of superior detection breadth. RIDER (secrets), TESLA (IaC), AI, MCP, severity recasting, and advanced report templates are not active in this pilot.

2. Requirements and operating systems

Dashboard: Docker Engine/Desktop with Linux containers and Docker Compose v2. It runs on Debian/Linux, macOS, or Windows; Debian is the initial production target. The Docker path does not require host Node or Go. Install the agent on the runner/developer host, not on the dashboard server.

Default ports are HTTP 8080 and PostgreSQL 5434; Compose binds both to host loopback. Change PANBOX_HOST_PORT or PANBOX_DB_HOST_PORT in .env if needed. Provide persistent storage for the PostgreSQL volume. PANBOX_BOOTSTRAP_PASSWORD is used only when the database is first initialized.

3. Local installation on Debian/Linux/macOS

From the PANBOX root:

```
sh scripts/init-local.sh
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
```

© 2026 BOXSPLOIT LABS

```
docker compose --env-file .env -f deploy/docker-compose.yml ps
curl -fsS http://127.0.0.1:8080/readyz
```

init-local.sh uses /dev/urandom, creates random secrets, restricts file permissions with umask 077, and refuses to overwrite .env. For a different port, set PANBOX_HOST_PORT=8092 before running it. Open http://127.0.0.1:8080 or your chosen port and sign in using PANBOX_BOOTSTRAP_EMAIL and PANBOX_BOOTSTRAP_PASSWORD from .env. Never send or commit that file.

Optional demo fixtures: PANBOX_SERVER=http://127.0.0.1:8080 bash scripts/seed-demo.sh. It needs curl and jq, prompts for the admin password, and loads two fictional repositories and four snapshots. Fixtures do not represent real vulnerabilities.

4. Local installation on Windows

Use Docker Desktop with Linux containers. In PowerShell at the PANBOX root:

```
./scripts/init-local.ps1
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
docker compose --env-file .env -f deploy/docker-compose.yml ps
Invoke-RestMethod http://127.0.0.1:8080/readyz
```

Run ./scripts/seed-demo.ps1 only if you want demo data. The Windows and Unix initializers write the same .env format; use only one for a new installation.

5. SCM repository, native agent, and CLI

In Repositories, choose GitHub, GitLab, Bitbucket, Azure Repos, Gitea, or Other Git, then enter that provider's stable repository ID or path. PANBOX assigns an internal repository ID; use that ID for --repo and CI variable PANBOX_REPOSITORY_ID. Create a repository-specific scan key and store it as PANBOX_TOKEN in CI secrets. Git remote text is not an authorization identity.

Download the agent archive for Linux, macOS, or Windows (amd64/arm64) from https://github.com/alterwriter/PANBOX/releases, verify it against SHA256SUMS, and put panbox/panbox.exe on the runner's PATH. The agent is one Go binary with built-in rules. ENETRON sends pinned package names and versions to the OSV API by default; use PANBOX_ADVISORY_URL or --advisory-url for an approved OSV-compatible mirror. No external scanner binary is installed.

```
panbox ready --server http://127.0.0.1:8080
panbox go --server http://127.0.0.1:8080 --repo "$PANBOX_REPOSITORY_ID" --path .
panbox shutdown
```

ready is optional. go supports --branch, --commit, --base-sha, --modules vaccin,enetron, --format text|json|sarif, and --offline --output panbox-results.json. Offline means no dashboard upload; ENETRON still contacts the advisory API unless you select only VACCIN. Exit codes: 0 pass, 1 gate failure, 2 incomplete scan/transport/configuration, 3 rejected authentication or permission. shutdown only clears temporary CLI cache.

6. Public SCM to private PANBOX

A public GitHub, GitLab, Bitbucket, Azure Repos, Gitea, or other Git repository can be scanned while PANBOX stays private without a public IP or domain. Three transport patterns are available:

- Local protected runner: the runner polls its SCM over outbound HTTPS and calls PANBOX on the local network. This supports a synchronous gate. GitHub Actions and GitLab examples are in deploy/github-actions-local-runner.yml and deploy/gitlab-ci-local-runner.yml.
- SSH relay: the PANBOX host opens a reverse tunnel to a reachable SSH relay; the shared runner opens a forward tunnel to that relay. PANBOX accepts no inbound internet traffic. This supports a synchronous gate;

deploy/gitlab-ci-ssh-relay.yml is a GitLab example of this provider-neutral path.

- Artifact pull: CI uploads a normalized bundle and a local PANBOX host downloads it, then imports it with panbox go --bundle panbox-results.json. This is asynchronous and cannot block CI. panbox relay-pull automates the GitLab Jobs API case; for other SCMs, use their artifact API or sync mechanism before the same bundle import.

Protect runners and branches, and keep scan keys out of untrusted pull request jobs. Pin SSH host keys and use restricted identities for a relay. Public source visibility does not guarantee public artifact access; use a scoped read token when needed.

7. RBAC and repository access

admin is the highest, immutable role. An admin can create custom roles with explicit permissions, add users, assign roles, and grant or revoke per-repository membership in Settings & Audit. Built-in roles include security_reviewer, developer, executive_viewer, and scanner_ci. Data access needs both an operation permission and access to the relevant repository. Custom roles cannot receive rbac:manage.

Pilot permissions: repos:read, repos:write, scans:read, scans:write, findings:read, reports:export, rbac:manage, tokens:manage, and audit:read. Test least-privilege users before production integrations. Role or membership changes constrain existing tokens on subsequent requests.

8. API, access, and scan keys

In Settings & Audit -> API & access keys, choose a label, type, expiry, scopes, and optionally a repository. API keys take selected integration scopes. Access keys are read-only. Scan keys are limited to scans:write and one repository. A secret is shown only once; store it in a secrets manager, not source control or .panbox.yml. Keys are hashed at rest, expiring, and revocable.

Read-only API test:

```
curl -H "Authorization: Bearer $PANBOX_ACCESS_KEY" http://127.0.0.1:8080/v1/repos
```

Browser login uses password, an HttpOnly session cookie, and CSRF token. api/openapi.yaml describes the pilot endpoints. To rotate a key, create a new one, update the client/CI secret, confirm access, then revoke the old one.

9. Dashboard, findings, gate, and reports

Overview shows repository coverage, severity counts, gate status, and scan timing. Repositories handles enrollment and keys; Findings filters by repository/branch/module/severity; Scans shows history and gate details; Reports exports technical CSV/JSON; Settings & Audit manages users, RBAC, keys, and events. Light and dark themes are available.

Standard severities remain critical, high, medium, low, info. ARK is the display label for Critical/High and BUG-STER for Medium/Low. Raw and effective severity fields are stored, but interactive recasting is not implemented yet. Partial scans are not clean; the dashboard retains the last successful scan. Technical exports require reports:export.

10. Operations, security, and resource sizing

Compose binds the server and PostgreSQL to 127.0.0.1. For access beyond the host, use an HTTPS reverse proxy with access controls or a private delivery path above; set PANBOX_SECURE_COOKIE=true when serving the browser over HTTPS. Back up the database/volume and test restoration before an important pilot. Do not expose PostgreSQL to the internet.

The pilot target is dashboard plus PostgreSQL on a 2 GB host, with one scan ingest at a time and no local AI model. A small Windows Docker Desktop fixture measured about 5.1 MiB server RAM and 37.3 MiB PostgreSQL RAM; 100

© 2026 BOXSPLOIT LABS

sequential API requests had a 7.1 ms median and 9.3 ms p95. This is not a Debian 2 GB guarantee; the agent on the runner needs separate sizing. Measure docker stats --no-stream and latency on the target host with representative data.

11. Troubleshooting

- /readyz fails: inspect docker compose --env-file .env -f deploy/docker-compose.yml ps and logs db server; confirm PostgreSQL health.
- Login fails after changing .env: the bootstrap password only applies at first database initialization. Do not delete the volume for a routine reset.
- Gate exits 2: check the supported manifest, advisory API, work path, endpoint, and runner logs. A partial scan is not a pass.
- Gate exits 3: check scan key expiry/revocation, scans:write scope, internal repository ID, role, and membership.
- A Docker runner cannot reach 127.0.0.1: its loopback is the container itself; use a reachable private host/container-network address.
- Artifact pull lags: inspect the timer, GITLAB_READ_TOKEN, allowed refs, Jobs API/artifact access, and relay state file.

12. Pilot limits and next steps

Available now: server/UI, repository/branch snapshots, built-in VACCIN source checks, built-in ENETRON dependency matching with OSV advisory data, gate, RBAC, keys, audit, technical CSV/JSON, and public-to-private connection examples for GitHub and GitLab plus a generic bundle import. Validate real SCM runners, OSV availability or an approved mirror, Debian load, backup/restore, TLS, and credential rotation in the target environment.

Detection coverage is narrow: selected source patterns and pinned npm/PyPI/Go manifests only. ENETRON needs network access to an advisory API and reports info when advisory severity is unavailable. RIDER, TESLA, severity recasting and approval, finding/executive PDF report templates, AI Security, MCP, OIDC, and API rate limiting are not available in this pilot. This PDF is a product guide, not PANBOX's report feature.

© 2026 BOXSPLOIT LABS